VtkAdaptor

```
mport vtk
from GeomBase import *
from Polyline import *

class VtkAdaptor:
    def __init__(self, bgClr = (0.95, 0.95, 0.95)):
        self.renderer = vtk.vtkRenderer()
        self.renderer.SetBackground(bgClr)
        self.window = vtk.vtkRenderWindow()
        self.window.AddRenderer(self.renderer)
        self.window.SetSize(900, 600)
        self.interactor = vtk.vtkRenderWindowInteractor()
        self.interactor.SetRenderWindow(self.window)
        self.interactor.SetInteractorStyle(vtk.vtkInteractorStyleTrackballCamera())
        self.interactor.Initialize()

    def display(self):
        self.interactor.Start()

    def setBackgroundColor(self, r, g, b):
        return self.renderer.SetBackground(r, g, b)

    def drawAxes(self, length = 100.0, shaftType = 0, cylinderRadius = 0.01, coneRadius =
0.2):
        axes = vtk.vtkAxesActor()
        axes.SetTotalLength(length, length, length)
        axes.SetShaftType(shaftType)
        axes.SetCylinderRadius(cylinderRadius)
        axes.SetConeRadius(coneRadius)
        axes.SetAxisLabels(0)
        self.renderer.AddActor(axes)
        return axes

    def drawActor(self, actor):
        self.renderer.AddActor(actor)
        return actor

    def drawPdSrc(self, pdSrc):
        mapper = vtk.vtkPolyDataMapper()
        mapper.SetInputConnection(pdSrc.GetOutputPort())
        actor = vtk.vtkActor()
```

```
        actor.SetMapper(mapper)
        return self.drawActor(actor)

    def drawSphere(self, r):
        src = vtk.vtkSphereSource()
        src.SetRadius(r)
        src.SetThetaResolution(50)
        src.SetPhiResolution(50)
        return self.drawPdSrc(src)


    def drawStlModel(self, stlFilePath):
        reader = vtk.vtkSTLReader()
        reader.SetFileName(stlFilePath)
        return self.drawPdSrc(reader)

    def removeActor(self, actor):
        self.renderer.RemoveActor(actor)

    def drawPoint(self, point, radius = 2.0):
        src = vtk.vtkSphereSource()
        src.SetCenter(point.x, point.y, point.z)
        src.SetRadius(radius)
        return self.drawPdSrc(src)

    def drawSegment(self, seg):
        src = vtk.vtkLineSource()
        src.SetPoint1(seg.A.x, seg.A.y, seg.A.z)
        src.SetPoint2(seg.B.x, seg.B.y, seg.B.z)
        return self.drawPdSrc(src)

    def drawPolyline(self, polyline):
        src = vtk.vtkLineSource()
        points = vtk.vtkPoints()
        for i in range(polyline.count()):
            pt = polyline.point(i)
            points.InsertNextPoint((pt.x, pt.y, pt.z))
        src.SetPoints(points)
        return self.drawPdSrc(src)

    def drawTriangles(self, triangles):
        cells = vtk.vtkCellArray()
        points = vtk.vtkPoints()
        for tri in triangles:
```

```
            A = tri.A
            B = tri.B
            C = tri.C
            idA = points.InsertNextPoint(A.x, A.y, A.z)
            idB = points.InsertNextPoint(B.x, B.y, B.z)
            idC = points.InsertNextPoint(C.x, C.y, C.z)
            vtkTri = vtk.vtkTriangle()
            vtkTri.GetPointIds().SetId(0, idA)
            vtkTri.GetPointIds().SetId(1, idB)
            vtkTri.GetPointIds().SetId(2, idC)
            cells.InsertNextCell(vtkTri)
        src = vtk.vtkPolyData()
        src.SetPoints(points)
        src.SetPolys(cells)
        mapper = vtk.vtkPolyDataMapper()
        mapper.SetInputData(src)
        actor = vtk.vtkActor()
        actor.SetMapper(mapper)
        return self.drawActor(actor)


    pass
```